به نام خدا

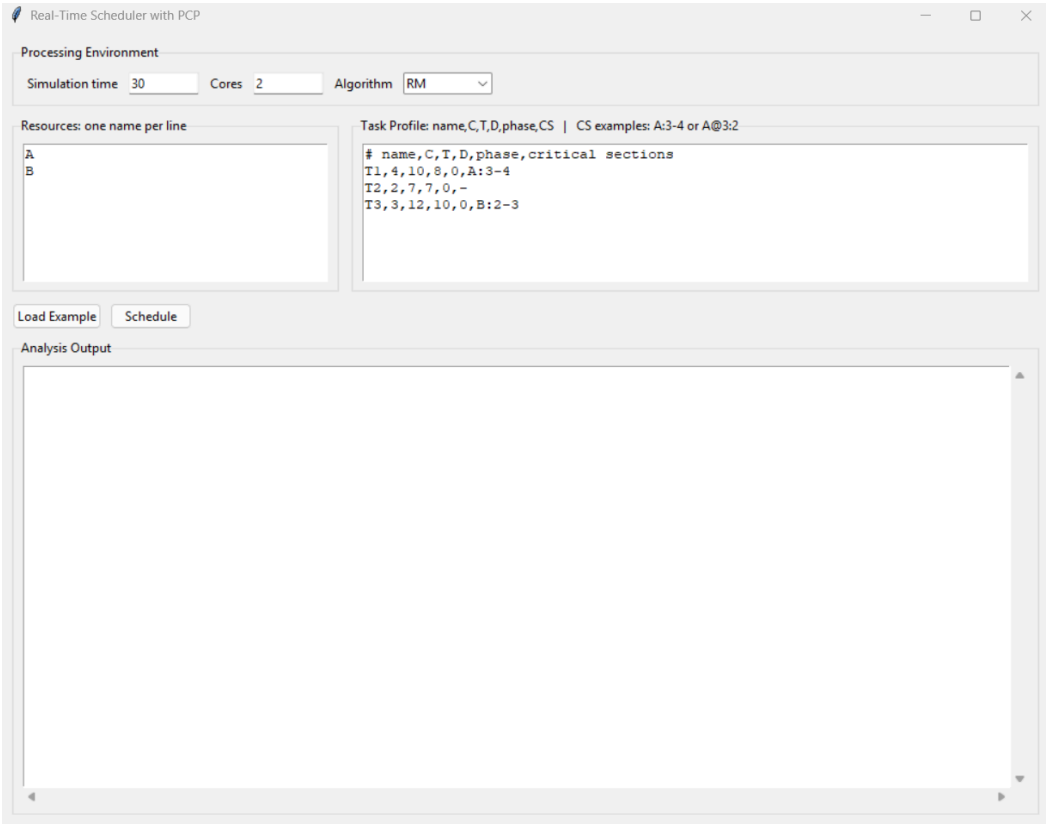
پژوهش اول سیستم‌های نهفته بی درنگ

مبینا خسروانی 402101628

صورت مسئله

در این تمرین از ما خواسته شده که یک scheduler با الگوریتم های RM یا EDF بسازیم. در این scheduler ما توانایی برنامه ریزی چند پروسسور با resource های مختلف را داریم که توسط task ها استفاده میشود. برای بهبود عملکرد در critical section ها از priority ceiling protocol استفاده میکنیم. در این پروتکل یک task با اولویت بالاتر فقط در حالتی می تواند یک تسک با اولویت پایین تر را preempt کند که اولویت این task از تمامی resource های اشغال شده بالاتر باشد.

ورودی برنامه



همانطور که در شکل بالا مشخص است قالب ورودی برنامه به این شکل است که زمان اجرای برنامه، تعداد پروسسور ها، الگوریتم مورد نظر در قسمت بالایی وارد میشود و در قسمت سمت چپ resource های مورد استفاده قرار میگیرد. در قسمت task profile مشخصات task ها یه صورت بالا میباشد در این مشخصات زمان اجرای هر تسک، دوره تناوب و ددلاین تسک قرار میگیرد. در این کد ما می توانیم برای هر task یک فاز اولیه تعریف کنیم که تسک ها در یک زمان شروع نشوند البته ما در ورودی نمونه فاز همه را برابر صفر قرار دادیم. Resource های مورد استفاده هر task به دو شکل زیر می توانند وارد شوند:

$$A: 3 - 4$$

یا :

$$A@3: 2$$

به این معنی که این task از resource A در step 3 و step 4 استفاده میکند یا در step 3 به مدت دو step در critical section قرار میگیرد.

چند core

برنامه قابلیت چند core دارد. در هر tick زمانی:

1. Job های جدید آزاد می‌شوند.
2. Job های آماده در ready queue قرار می‌گیرند.
3. براساس RM یا EDF مرتب می‌شوند.
4. به تعداد coreها job انتخاب می‌شود.
5. هر job در یک واحد زمانی روی یک core اجرا می‌شود.

مدل استفاده شده global scheduling است، یعنی همه job ها در یک ready queue مشترک هستند. یک job همزمان روی دو core اجرا نمی‌شود.

الگوریتم RM

در Rate Monotonic یا RM اولویت‌ها ثابت هستند. هر task که period کوچک‌تری دارد، اولویت بالاتری دارد:

$$T_i < T_j \rightarrow Priority_i > Priority_j$$

مثلا اگر:

$$T_1 = 10\,,\ T_2 = 7,\ T_3 = 12$$

اولویت task دو بیشتر از یک و بیشتر از سه میباشد.

مزیت RM این است که ساده، قابل تحلیل و مناسب سیستم‌های periodic است. محدودیت آن این است که همیشه از کل ظرفیت پردازنده استفاده بهینه نمی‌کند.

الگوریتم EDF

در Earliest Deadline First اولویت job ها dynamic است. در هر لحظه job ی که deadline مطلق نزدیک‌تری دارد زودتر اجرا می‌شود:

$$d_i < d_j \rightarrow Priority_i > Priority_j$$

در EDF ممکن است دو job از یک task در زمان‌های مختلف اولویت متفاوتی داشته باشند، چون deadline مطلق آن‌ها متفاوت است.
برای سیستم تک‌پردازنده preemptive و taskهای مستقل، EDF در حالت deadline برابر period شرط مهم زیر را دارد:

$$U = \sum \frac{C_i}{T_i} \leq 1$$

برای deadline های عمومی‌تر از Demand Bound Function استفاده می‌شود.

Demand Bound Function برای EDF

برای EDF، به خصوص وقتی deadline ها عمومی باشند، از تابع demand bound استفاده می‌شود:

```
def demand_bound_rows(tasks: list[Task], limit: int, processors: int = 1)
:[[-> list[tuple[int, int, int, bool
{points = {0
:for task in tasks
k = 0
:while k * task.period + task.deadline <= limit
(points.add(k * task.period + task.deadline
k += 1
[] = rows
:(for t in sorted(points
demand = 0
:for task in tasks
jobs_due = max(0, math.floor((t - task.deadline) /
(task.period) + 1
demand += jobs_due * task.wcet
capacity = processors * t
```

```
((rows.append((t, demand, capacity, demand <= capacity
return rows
```

این تابع نشان می‌دهد تا زمان t چه مقدار کار حتما باید کامل شده باشد.
برای یک core شرط:

$$dbf(t) <= t$$

برای m core، ظرفیت بازه برابر:

$$dbf(t) <= m * t$$

مثلا در نمونه برای t = 10:

برای T1:

$$floor((10 - 8)/10) + 1 = 1, demand_1 = 1 \times 4 = 4$$

برای T2:

$$floor((10 - 7)/7) + 1 = 1, demand_2 = 1 \times 2 = 2$$

برای T3:

$$floor((10 - 10)/12) + 1 = 1, demand_3 = 1 \times 3 = 3$$

پس:

$$dbf(10) = 4 + 2 + 3 = 9$$

برای دو core:

$$m \times t = 2 \times 10 = 20 , 9 <= 20$$

پس در این نقطه demand از ظرفیت بیشتر نیست.

Response Time Analysis

برای الگوریتم‌های fixed priority مثل RM، زمان پاسخ بدترین حالت task با رابطه زیر محاسبه می‌شود:

$$R_i = C_i + B_i + \sum_{j<i} ceil(\frac{R_i}{T_i}) C_j$$

که در آن:

- B_i: زمان blocking ناشی از critical section ها
- C_i: زمان اجرای task
- R_i: زمان پاسخ worst case
-

محاسبه به صورت تکراری انجام می‌شود:

$$R_i^{(0)} = C_i + B_i$$

اگر تکرار به مقدار ثابت برسد و:

$$R_i <= D_i$$

باشد، task قابل زمان‌بندی است.

کد استفاده شده برای این محاسبه به این شکل است:

```
)def response_time_analysis
,[tasks: list[Task
,[priority_rank: dict[str, int
,[blocking_terms: dict[str, int
:[[list[dict[str, object <- (
([ordered = sorted(tasks, key=lambda task: priority_rank[task.name
[] = [[rows: list[dict[str, object
:(for index, task in enumerate(ordered
[higher_priority = ordered[:index
[blocking = blocking_terms[task.name
current = task.wcet + blocking
[iterations = [current
converged = True
:(for _ in range(100
)interference = sum
```

```
math.ceil(current / hp_task.period) * hp_task.wcet
for hp_task in higher_priority
(
next_value = task.wcet + blocking + interference
(iterations.append(next_value
:if next_value == current
break
:if next_value > 10_000
converged = False
current = next_value
break
current = next_value
)rows.append
}
,task": task.name"
,"-" hp": ", ".join(hp.name for hp in higher_priority) or"
,blocking": blocking"
,iterations": iterations"
,response": current"
,deadline": task.deadline"
,ok": converged and current <= task.deadline"
{
(
return rows
```

محاسبه Blocking در PCP

در PCP، blocking برای task i از critical section های taskهای پایین اولویت ایجاد می‌شود. اگر یک task پایین اولویت resource ای را بگیرد که ceiling آن می‌تواند task i را block کند، طول آن critical section در B_i اثر دارد.

کد مورد استفاده برای این محاسبه:

```
)def compute_blocking_terms

,[tasks: list[Task

,[priority_rank: dict[str, int
```

```
,[resource_ceilings: dict[str, int
:[dict[str, int <- (
{} = [blocking_terms: dict[str, int
:for task in tasks
[task_priority = priority_rank[task.name
blocking = 0
:for lower_task in tasks
:if priority_rank[lower_task.name] <= task_priority
continue
:for section in lower_task.critical_sections
:if resource_ceilings[section.resource] <= task_priority
(blocking = max(blocking, section.length
blocking_terms[task.name] = blocking
return blocking_terms
```

ورودی نمونه برنامه:

```
Cores = 2
Simulation time = 30
Algorithm = RM

:Resources
A
B
```

```
:Tasks
T1,4,10,8,0,A:3-4
-,T2,2,7,7,0
T3,3,12,10,0,B:2-3
```

خروجی برنامه نمونه:

===== Input Summary =====

Algorithm: RM

Cores: 2

Simulation time: 30

Resources: A, B

===== Necessary and Sufficient Bounds =====

(U = sum(C_i/T_i) = 0.9357 (93.57%

Necessary utilization bound for m cores: U <= m -> 0.9357 <= 2 : PASS

RMA Liu-Layland sufficient bound for n=3: n(2^(1/n)-1) = 0.7798

RMA sufficient test on one core: U <= bound -> NO GUARANTEE

.EDF demand-bound check shown up to t=30; condition dbf(t) <= m*t

===== Priority Ceiling Protocol =====

Priority rank: T2:1, T1:2, T3:3

Resource ceilings: A:2, B:3

Blocking terms B_i: T1:0, T2:0, T3:0

===== Exact Response Times from Simulation =====

Task | Completed Jobs | Max Exact R | D | All R values | Result

T1 | 3 | 4 | 8 | [4, 4, 4] | OK

T2 | 5 | 2 | 7 | [2, 2, 2, 2, 2] | OK

T3 | 3 | 7 | 10 | [7, 5, 3] | OK

===== Response-Time Analysis with Blocking =====

Task | hp(i) | B_i | iterations | R_i | D_i | Result

T2 | - | 0 | 2 -> 2 | 2 | 7 | Schedulable

T1 | T2 | 0 | 4 -> 6 -> 6 | 6 | 8 | Schedulable

T3 | T2, T1 | 0 | 3 -> 9 -> 11 -> 15 -> 17 -> 17 | 17 | 10 | Not guaranteed

===== Demand Bound Function =====

t | dbf(t) | m*t | Result

OK | 0 | 0 | 0

OK | 14 | 2 | 7

OK | 16 | 6 | 8

OK | 20 | 9 | 10

OK | 28 | 11 | 14

OK | 36 | 15 | 18

OK | 42 | 17 | 21

OK | 44 | 20 | 22

OK | 56 | 26 | 28

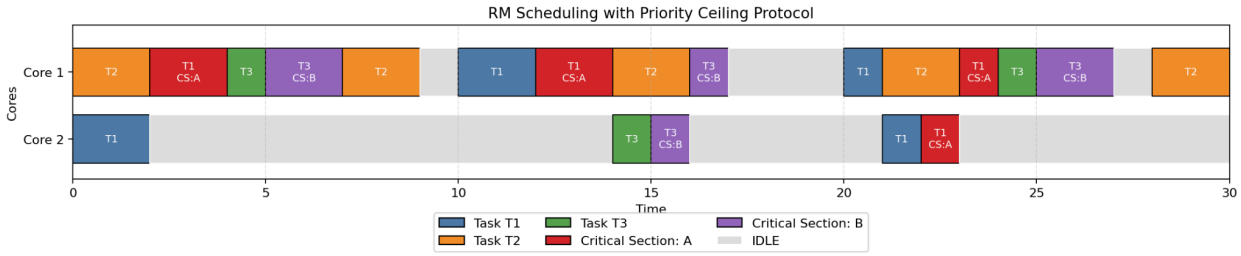
===== Simulation Summary =====

Completed jobs: 11

Deadline misses: 0

Busy slots: 31/60

Observed CPU utilization: 51.67%



در این شبیه سازی deadline miss رخ نداده است. با این حال، چون utilization از کران کافی RM بزرگ‌تر است، تست Liu Layland به تنهایی تضمین نمی‌دهد. این تفاوت مهم است: تست کافی اگر fail شود، یعنی تضمین ساده نداریم، نه اینکه حتما deadline miss رخ می‌دهد.